

PREDATOR RF

Operator Guide

Pick this up cold. Zero to a working RF picture in under 30 minutes.

The single document. If the prior operator is unavailable, this is everything you need to pick the system up cold and operate with it.

Rev 2 — June 2026 · KrakenSDR DF, sensor SA beacon, CoT type corrections

Table of Contents

1. What this is, in one paragraph
2. Two deployment paths
3. Minimum kit
4. First-time install (sideload)
5. Plug in the radio and start receiving
6. The eight tabs
7. Mission modes
8. The five day-one workflows
9. Editing fields on a touchscreen
10. The status bar
11. Kujhad fleet — protocol, pairing, TLS pinning
12. Geolocation — TDOA, error ellipse, confidence
13. Direction finding with KrakenSDR — LOB, crosscut, map layers
14. Track lifecycle
15. Trust model
16. The intelligence layer
17. ATAK / TAK CoT integration
18. AutoTasker
19. Operator overrides
20. Mission lifecycle
21. Path 2: the Python backend
22. Hardware capability table
23. Field-day checklist
24. Troubleshooting
25. Bill of materials
26. Glossary
27. Quick reference card

1. What this is, in one paragraph

Predator RF is a phone-first software-defined-radio cockpit for a solo SIGINT operator. You plug a USB SDR (a small dongle that acts as a radio receiver) into an Android phone or a Linux box, point an antenna at the world, and the app shows you what's transmitting in real time: frequency, signal power, GPS-stamped location, and whether you've seen it before. It records baselines so the next sweep flags only what's *new* in the area. It automates band scans with target and exclude lists, links to other phones or Raspberry Pi sensors as a private fleet (called **Kujhad**), geolocates transmitters by time-difference-of-arrival (TDOA) or KrakenSDR direction-finding (LOB) when you have GPS-synchronized nodes, decodes P25 / RTL433 / POCSAG / ADS-B / AIS, and pushes selected detections to ATAK / TAK as Cursor-on-Target (CoT) alerts. The whole system is **receive-only** — there is no transmit path anywhere. Every output (CoT export,

AutoTasker re-tunes) requires the operator to explicitly arm it. Built on the SDR++ core. Sideloads as an APK; the Python backend ships as a systemd service.

2. Two deployment paths (you can run one or both)

Path 1 — Phone (Android)

The C++ Predator RF app on a Galaxy S22 (or any Android 10+ device), driving a USB-C SDR through an OTG cable. Standalone — no other infrastructure needed. The phone IS the cockpit. Optionally peers with other phones or RPi sensors via Kujhad over Wi-Fi or a private overlay (ZeroTier / Tailscale). KrakenSDR DF is available via WebSocket to a companion RPi.

Path 2 — Linux TOC + RPi sensor nodes (Python backend)

A Python backend service (predator-rf.service) runs on a Linux operator workstation or Raspberry Pi. It:

- Owns persistence (SQLite mission ledger — a local database that logs everything).
- Aggregates events from one or more Kujhad-equipped sensor nodes (phones, RPi-with-SDR, KrakenSDR arrays) via the Kujhad HTTP API.
- Runs the TDOA coordinator, LOB triangulator, decision engine, AutoTasker, and CoT emitter centrally.
- Exposes a REST + SSE API on port 8000 (token-protected) for dashboards or chaining into a higher TOC (CoC mode).

Tip: The two paths mix freely. A typical mission: one Linux workstation running the backend, three phones in the field sharing their picture into Kujhad, one or two RPi sensors at fixed points, and a KrakenSDR array for direction finding. All fuses into one operator screen.

3. Minimum kit (you need ALL of these)

Item	Why you need it
Android phone, Android 10 (API 28) or newer	Runs the app
USB-C OTG cable or adapter (USB-C → USB-A female)	Connects the SDR to the phone
USB SDR — RTL-SDR Blog v4 (\$40) minimum; HackRF One (\$340) is the upgrade	The actual radio hardware. Without this, there is nothing to receive.
An antenna — even the rubber duck shipped with the SDR works to start	Without one you receive nothing
The Predator RF APK	Built from the GitHub repo, or supplied by your team

No internet required after install. No subscription. No account.

A full tiered bill of materials (cheap → full fleet) is in § 25.

4. First-time install (sideload)

Predator RF is not on the Play Store. Install it manually by 'sideloading':

1. **Unlock developer options:** Settings → About phone → tap *Build number* seven times until it says 'Developer mode is ON'.
2. Settings → Developer options → turn on *Install via USB* and *USB debugging*.
3. Get the APK file onto the phone (USB cable from a computer, Google Drive, email — anything that puts the file in your Downloads folder).
4. On the phone: open the Files app → Downloads → tap app-debug.apk.
5. When Android asks 'Install unknown apps?' → Allow for whatever app you used to open it → Install.
6. Open Predator RF.

When the app first launches it will ask for permissions — **grant all of them:**

- **Location ('While using the app')** — required for GPS-stamped hits and TDOA.
- **Storage / Files** — required for saving baselines and exporting CSVs.
- **USB device access** — pops up the first time you plug in your SDR. Tap *Always allow for this device*.

If you skip any permission: the app still runs, but the matching feature goes completely silent — no map fix, no exports, no SDR.

5. Plug in the radio and start receiving

1. Connect your SDR to the phone with the USB-C OTG adapter. Antenna goes on the SDR.
2. The first time you plug in, a system dialog pops: 'Open Predator RF when this USB device is connected?' — check the box and tap OK.
3. In the app, tap the **source dropdown** (top-left, says None until a radio is selected) → pick the type matching your SDR (RTL-SDR, Airspy, HackRF, etc.).
4. Tap ■ **Start Listening**.
5. The waterfall (the scrolling coloured display) starts moving. You're live.

Waterfall stays black? See § 24 Troubleshooting.

6. The eight tabs (left-side rail, top to bottom)

Code	Name	What it's for
SPEC	Spectrum	Live waterfall + tuner. Where you actually look at signals.
HITS	Hits & Events	Every signal the app has noticed, plus the running event log.
NET	Network	Catalog of known networks / talkgroups / aliases — your rolodex of emitters.
MAP	Map	GPS-stamped hits on a map, with TDOA fix dots, LOB bearing wedges, error ellipses, and node positions.
MIS	Mission Config	Mission mode (Manual / Classify / Scan / QuickScan), search bands, targets, excludes, dwell timing.
KUJ	Kujhad Fleet	Link this phone to other Predator RF phones or Raspberry Pi sensors over a network.
SYS	System	App settings — modules, themes, decoder bridges, ATAK/CoT, baseline comparison, TLS fingerprint.

BASE	Baseline	Record normal noise floor, save it, suppress it so only NEW signals fire as hits.
------	----------	---

7. Mission modes (set on the MIS tab)

Mode	What it does	When to pick it
Manual	Direct operator tuning and marker ownership. Nothing automated.	Free recon, training, demonstration.
Classify	You stay in manual control while idle resources watch the band — passively classifying whatever crosses the threshold.	When you're working a known signal but want background awareness.
Scan	Automated search-and-target workflow across all configured bands with full target/exclude logic.	The default field mode for a real operation.
QuickScan	Rapid single-marker sweep for a quick area check — no target persistence.	Walk into a new area, spend 60 s seeing what's hot, decide whether to stay.

8. The five day-one workflows

8.1 Just look around (passive recon)

1. SPEC tab → set the source → ■ Start Listening.
2. Drag the waterfall left/right to retune. Pinch to zoom.
3. Tap a peak to drop a marker on it.
4. To name it: tap the marker → Assign Marker → tap 'Tap to edit' in the popup → type a label.

8.2 Mark a hit and send it to your team

1. With a marker on a signal, tap *Assign Marker* if it isn't already named.
2. The marker becomes a hit — visible on the HITS tab.
3. To push to ATAK / TAK: SYS tab → ATAK / CoT section → enter TAK server IP + port → Enable CoT.

8.3 Record a baseline so you only see NEW signals

1. BASE tab → Frequency Ranges → set Range Name, Start Hz, Stop Hz → + **Add Range**. Repeat per band.
2. Recording section → set a filename → tap ■ **START RECORDING** (green).
3. Let it run for at least 5 minutes (30 min for a thorough sweep).
4. ■ **STOP RECORDING** → Save to File.
5. SYS tab → Baseline Comparison → load the file → enable *Scan against baseline*. Set a threshold (default 6 dB).

8.4 Run an automated scan across multiple bands

1. MIS tab → Mission Mode → **Scan**.
2. Search Bands → add bands (Name + Start Hz + Stop Hz) → + **Add Band**.

3. Targets — frequencies to flag as priority (optional).
4. Excludes — frequencies to ignore (your own gear, known broadcast).
5. Back to SPEC → ■ Start Listening. The app sweeps and drops hits as it finds them.

8.5 Link a second phone or RPi sensor

See § 11 for the full walkthrough. Two-minute version:

- **On the device to publish from:** KUJ → Listen → set port + Device name + API key → toggle Listen ON.
- **On the operator phone:** KUJ → Add Peer → enter Name, Host (IP), Port, API key.

9. Editing fields on a touchscreen

1. Tap the field. A popup opens at the *top* of the screen, above the keyboard.
2. The keyboard slides up automatically.
3. Type. The popup is pinned high so the keyboard never covers it.
4. Tap **OK** (or hit Enter) to commit. Cancel to discard.

Letters not appearing? Some composing / autocorrect-heavy keyboards don't work inside the NativeActivity engine. Switch your keyboard to a non-composing one (e.g. Google Keyboard with autocorrect off) for the edit. Numeric input always works regardless of keyboard.

10. The status bar at the top

Indicator	Colour	What it means
Thermal	Green / dim grey	Phone temperature is fine
	Orange	SoC heating up; expect frame rate drops
	Red	Severe throttling — lower sample rate, get out of direct sun
GPS	Green	Lock acquired, position fresh (age < 60 s)
	Yellow	Lock but ageing — TDOA will refuse this node soon
	Red / dim	No lock — TDOA disabled, map uses last known position
Kujhad	Green	All peers connected
	Yellow	Some peers down
	Red	None reachable
CoT	Off / dim	CoT export disabled
	Green	Enabled, packets going out
	Yellow	Enabled but holding at the manual-approval queue

11. Kujhad fleet — protocol, pairing, TLS pinning

11.1 What it is

Kujhad is the private peer protocol that links Predator RF instances together. Each instance can be a **Device** (publishes its RF state and event stream) and/or a **Controller** (connects to Devices and mirrors their state). Hub-and-spoke — you pick which devices to see. Think of it as a secure private radio network for your sensor nodes.

11.2 The wire protocol

HTTP/1.1 + JSON, with a single API key in the X-Kujhad-Key header. Default port **41947**.

Endpoint	Purpose
GET /v1/identify	Device name, version, role, hardware profile
GET /v1/gps	Current GPS fix
GET /v1/state	Mission mode, scan status, threshold, search bands, decoder roster
GET /v1/events?since=N	Hit / decoded-event stream since serial N
GET /v1/timing	Clock source, PPS lock, offset, drift, last-sync age
POST /v1/command	Issue a typed command: tune, scan, mission, identify

Hard safety boundary: Any command in the tx class is rejected unconditionally by the dispatcher. The whole module never opens a transmit path.

11.3 Make this device discoverable (publish)

1. KUJ tab → Listen section.
2. Listen port — leave at default (41947) unless you have a port conflict.
3. Device name — short identifier (e.g. alpha-truck, bravo-roof).
4. API key — leave the auto-generated 32-hex-char key, or paste your own.
5. Toggle **Listen ON**.

11.4 Pair to a peer (subscribe)

1. KUJ tab → Add Peer.
2. Name — anything memorable.
3. Host — the publisher's IP address.
4. Port — the publisher's listen port.
5. API key — must match the publisher exactly, character-for-character.
6. TLS — leave OFF unless you've armed TLS on the publisher (see § 11.5).
7. Tap **Add peer**.

11.5 TLS pinning (optional)

On the publisher: SYS → Kujhad TLS → Generate self-signed cert → write down the SHA-256 fingerprint → read it to the controller out-of-band (voice, paper — NOT the same network you're pairing on) → toggle TLS enabled.

On the controller: KUJ → Add Peer → toggle TLS → paste the fingerprint into *Pinned fingerprint*. Mismatch on first connect = abort with loud warning.

11.6 Python backend → Kujhad device

```
FLEET_NODES=alpha@192.168.1.10:41947:<api_key>;hackrf,bravo@192.168.1.11:41947:<api_key2>;rtlsdr
```

Format per node: node_id@host:port:api_key:hardware_code. The backend polls each node: events at 1 Hz, GPS at 1 Hz, full state every 5 s, timing telemetry every 30 s.

11.7 Kujhad troubleshooting

Symptom	Most likely cause	Fix
Peer red / never green	Network unreachable, port blocked, or API key mismatch	Ping the host; telnet the port; verify the key character-for-character
Peer green but no events	Mission mode is Manual on the peer	Change peer to Scan or Classify
401 on every request	API key wrong	Re-copy the key
TLS handshake failure	Wrong fingerprint pinned, or peer cert regenerated	Re-pin the new fingerprint after verifying out-of-band

12. Geolocation — TDOA, the error ellipse, what confidence actually means

12.1 What TDOA is

TDOA stands for Time-Difference-of-Arrival. When two or more GPS-synchronized sensor nodes hear the same transmission, the tiny difference in *when* each node received it (multiplied by the speed of light) gives a curved line where the transmitter must be. With three or more nodes those curves intersect at a position fix.

Think of it as triangulation using time rather than angle — two nodes gives you a search arc, three nodes gives you a point on the map.

12.2 What you need for a fix

Requirement	Why it matters
≥ 2 distinct sensor nodes	Two readings from the same receiver give no time-difference information
GPS lock on each participating node	Without a position you can't do the geometry at all
GPS lock age < 60 s on each node	A stale GPS fix means the node may have moved — the math breaks silently
Each node's hearings within a 5-second window	TDOA assumes one transmission; measurements too far apart in time can't be compared
A common time reference	GPSSDO/PPS-disciplined hardware → high trust; system-clock-only → low trust but still produces a fix

Inclusive by policy: Any GPS-equipped node participates, even cheap RTL-SDRs without a GPSSDO. Their timing trust is capped at 0.5 instead of 0.98, and the fix confidence is scaled accordingly. A rough fix from four cheap nodes is still useful — it gives you a search area instead of nothing.

12.3 The error ellipse on the map

- Base radius scales as $50 \text{ m} + (1 - \text{confidence}) \times 4950 \text{ m}$ — high-confidence fix shrinks toward 50 m, zero-confidence fix grows toward 5 km.

- Eccentricity comes from your node geometry: clustered nodes → near-circular blob; nodes in a line → long thin ellipse perpendicular to the baseline.
- Rotation follows the principal axis of the node cluster.

12.4 What 'confidence' actually means in three places

Where you see it	What it actually measures
track.confidence on the HITS row	Detection confidence — how sure the system is this is a real, persistent emitter. Starts at 0.1, climbs with corroborating observations.
track.location_confidence (drives the map ellipse)	Geolocation confidence — TDOA or LOB fix quality only. Independent of detection confidence.
assessment.confidence on the threat assessment	Assessment confidence — the intelligence layer's confidence in its threat level call. Independent of both above.

12.5 RSSI proximity fallback (single-node)

This is NOT real geolocation. Treat the circle as a search area, not a confirmed position. Never report these coordinates to higher echelon as an emitter fix.

When `RSSI_PROXIMITY_ENABLED=true`, single-node tracks get a coarse position estimate centred on your phone's GPS. The circle radius is estimated from the signal's power via free-space path-loss model. As soon as a second GPS-synced node hears the same emitter, location method flips to TDOA and the proximity circle is replaced by the proper ellipse.

13. Direction finding with KrakenSDR — LOB, crosscut, map layers

13.1 What KrakenSDR adds

KrakenSDR is a five-channel coherent software-defined radio built for Direction Finding (DF). While TDOA needs two or more nodes to hear the same signal, a *single* KrakenSDR array can produce a **Line of Bearing (LOB)** — an angular direction toward the transmitter from the array's position. Two arrays crossing their LOBs produces a **crosscut fix**, placed on the map. Predator RF integrates KrakenSDR as a first-class location source alongside TDOA.

One KrakenSDR array = a bearing wedge on the map. Two arrays crossing = a crosscut dot. TDOA always wins if it produces a fix first, but LOB is operationally useful with just one array and no TDOA nodes in range.

13.2 The hardware pipeline

The KrakenSDR hardware is a five-input coherent SDR array (five RTL-SDR chips sharing a reference clock). It connects to a companion Raspberry Pi that runs two processes:

Process	Port	Role
daq_fw	5000	Firmware bridge — coherent IQ from the five ADCs
krakensdr_doa	8082 (WebSocket /ws)	DOA solver — streams JSON <code>doa_result</code> messages with bearing, confidence, SNR

Predator RF connects to `krakensdr_doa` over WebSocket — it does *not* need the IQ data, only the computed bearings. The array hardware never needs to be directly reachable from the operator's phone or workstation; only the WebSocket port on the Pi needs to be.

13.3 What the WebSocket stream looks like

```
{
  "type":      "doa_result",
  "freq_hz":   433920000,
  "bearing_deg": 127.5,
  "bearing_std_deg": 5.2,
  "confidence": 0.83,
  "power_dbfs": -42.1,
  "snr_db":    12.3,
  "gps_lat":    37.4,
  "gps_lon":    -122.1,
  "heading_deg": 0.0,
  "timestamp_unix": 1718035200.123,
  "node_id":    "kraken-0"
}
```

Messages with any type other than 'doa_result' are silently ignored. A measurement is accepted only when bearing_deg is in [0, 360), gps_lat and gps_lon are both present and non-zero, and confidence is in [0, 1].

13.4 Setting up the RPi (one-time)

Start the DOA engine on the KrakenSDR RPi:

```
cd ~/krakensdr_doa
./kraken_doa_start.sh
```

Or use the Predator RF RPi install script which handles everything automatically:

```
sudo bash deploy/install_rpi.sh --kraken
```

The `--kraken` flag additionally:

- Installs numpy and scipy for the N-LOB solver path.
- Installs rtl-sdr, libatlas-base-dev, usbutils via apt.
- Writes `/etc/udev/rules.d/99-krakensdr.rules` granting USB access.
- Clones `krakensdr_doa` into `/opt/krakensdr_doa` (shallow clone of the official GitHub repo) and pip-installs its requirements.
- Writes `/etc/systemd/system/krakensdr-doa.service` (enabled, not started) and `/etc/krakensdr/predator.env` with `KRAKEN_DOA_HOST/PORT` tunable hints.

First-start sequence after `--kraken`:

```
# 1. Confirm the array is plugged in
lsusb | grep 0bda

# 2. Review / adjust the env file
sudoedit /etc/krakensdr/predator.env

# 3. Start the DOA engine
sudo systemctl start krakensdr-doa

# 4. Watch its log
journalctl -u krakensdr-doa -f
```

13.5 Connecting Predator RF to KrakenSDR (the SDRPP module)

In the Predator RF app (or desktop build), the **kraken_lob_decoder** module bridges the WebSocket stream into the app's hit pipeline.

1. Open Module Manager (SYS tab on Android / Module Manager on desktop).
2. Enable **kraken_lob_decoder**.
3. In the module panel, set:

Setting	Default	What to set it to
Host	127.0.0.1	IP of the machine running krakensdr_doa
Port	8082	krakensdr_doa WebSocket port
Path	/ws	WebSocket path — leave as-is
Node ID	kraken-0	Device ID used when the JSON node_id field is empty

A green ● **Connected** indicator confirms the WebSocket link is live. A **krakensdr_source** information panel is also available in the source selector — it displays an architecture overview but does not produce IQ data (KrakenSDR raw IQ is not accessible to SDR++).

Android note: KrakenSDR over USB is not supported on Android. The array must run krakensdr_doa on a companion RPi reachable over Wi-Fi or LAN from the Android device. The WebSocket bridge (kraken_lob_decoder) works the same way on Android as on Linux.

13.6 How LOB becomes a fix on the map

Each accepted doa_result message becomes a **LOBMeasurement** — one bearing observation from one node. The **LOBTriangulator** (in backend/fusion/lob_triangular.py) decides what to do with the queue of measurements for each emitter track:

# of nodes with bearings	What happens
0 or 1	No fix — need at least two crossing lines to get a position
2	Closed-form 2-line intersection. Vetoed when crossing angle < 15° (nearly parallel lines give huge uncertainty). Returns a crosscut dot + error radius.
3+	scipy.optimize.least_squares (Levenberg-Marquardt) preferred; numpy WLS fallback; 2-LOB closed-form last resort. More nodes = smaller error radius.

Measurements older than 30 seconds are excluded before the solver runs (TTL window), so a bearing from an earlier sweep can't distort the current fix.

Confidence ceiling: LOB confidence is capped at 0.70 (MAX_LOB_CONFIDENCE). DOA phase ambiguity limits confidence even on a perfect five-node crossing — this is intentional. TDOA always wins if it produces a fix for the same emitter.

13.7 The map layers

Layer	Toggle	Geometry	Colour	What to look for
-------	--------	----------	--------	------------------

LOB Wedges	DF / LOB → LOB Wedges	Bearing fan polygon from node, ±uncertainty, 25 km range	Matches threat level palette	The direction the array is pointing you — the emitter is somewhere along this wedge
LOB Crosscut	DF / LOB → LOB Crosscut	Filled circle at crosscut point	Magenta (distinct from TDOA cyan and RSSI yellow)	The estimated emitter position from two crossing LOBs

When a crosscut exists AND a TDOA fix exists for the same emitter, the location method becomes **lob_tdoa_hybrid** — the two fixes are blended using inverse-variance weighting ($w = 1/r^2$), producing a tighter ellipse than either method alone. The hybrid confidence is capped at 0.85.

13.8 CoT export with LOB

When a track has an LOB bearing, the CoT export injects a <sensor> element into the CoT detail block:

```
<sensor bearing="127.50" fov="10.4" range="25000"
  type="LOB" model="KrakenSDR" nodes="2"/>
```

ATAK / WinTAK render this as a bearing spoke with a field-of-view wedge overlay on the tactical picture. The CoT type depends on whether a crosscut fix is available:

- **Crosscut fix present:** CoT type a-u-G (unknown ground unit) — emitter marker at the crosscut position.
- **Bearing only (no crosscut):** CoT type b-m-p-s-p-loc (point of interest) — marker placed at the array (node) position, not the emitter.

13.9 Build flags

KrakenSDR modules are **ON by default** in both Linux and Android builds.

```
# Linux — exclude KrakenSDR:
./build_linux.sh --no-kraken

# Android — always included; enable at runtime via Module Manager

# CMake (explicit):
-DOPT_BUILD_KRAKENSDR_LOB_DECODER=OFF
-DOPT_BUILD_KRAKENSDR_SOURCE=OFF
```

13.10 Footguns — read these before you go in the field

#	Footgun	What to do
1	Heading correction is the caller's responsibility. For vehicle-mounted arrays, heading_deg must be set to the vehicle's compass heading so Predator RF can interpret bearing_deg correctly. Predator RF does NOT apply the heading correction internally — bearing_deg must be true-north on arrival at the backend.	Static arrays: set heading_deg=0 and you're done. Vehicle-mounted: add the vehicle heading to the raw DOA bearing (mod 360) before the doa_result message leaves krakensdr_doa, or the fix will be wrong by exactly your vehicle's heading offset.

2	Crossing-angle veto. When two bearing lines are nearly parallel (crossing angle < 15°), the 2-node fix is suppressed — geometrically the crosscut could be anywhere along both lines.	Wait for the emitter or the array to move so the angle widens, or wait for a third node to join.
3	25 kHz frequency association window. ingest_lob() matches bearings to tracks within ±25 kHz. Two emitters within 25 kHz of each other can get each other's LOB attached.	Inject a custom LOBTriangulator with a tighter freq_tolerance_hz into TrackManager.__init__, or manually merge/split tracks on the HITS tab.
4	numpy is optional but required for N-LOB WLS. Without numpy, the triangulator falls back to the 2-LOB closed-form solver using only the first two measurements even if you have five nodes.	Install numpy scipy into the backend venv: pip install numpy scipy. The --kraken flag on install_rpi.sh does this automatically.
5	Android KrakenSDR over USB is not implemented. The Android build includes the kraken_lob_decoder WebSocket module but not direct USB access.	The array must run krakensdr_doa on a companion RPi reachable over Wi-Fi/LAN. The WebSocket works the same way.

14. Track lifecycle — NEW → TRACKING → STABLE → COASTING → LOST

State	Triggered by	What this means for you
NEW	First detection	One sighting, not yet corroborated. Don't act on it alone.
TRACKING	≥ 3 observations	The system believes this is real. Scoring is live. Time to pay attention.
STABLE	≥ 10 observations	Well-characterised. AutoTasker may auto-task nodes here. High confidence.
COASTING	Track aged out of last-seen window but within replay window	'Was here, hasn't been heard in a while, expected to return.' Still in the live picture.
LOST	Beyond the replay window	Archived. Removed from the live picture but stays in the mission ledger.

The state appears on the HITS row and gates how the system behaves. AutoTasker and CoT escalation are both more conservative on NEW tracks.

15. Trust model — node trust score, timing trust, sensitivity trust

Every sensor node carries a composite trust score between 0.05 and 0.98. This score multiplies the weight of that node's observations during fusion, so a flaky cheap node doesn't drag down a high-quality fix.

```
operational = base_trust × uptime_fraction × (1 - false_positive_rate)
multi_node  = multi_node_agreement × 0.2
```

```

hardware      = freq_stability × 0.3
               + sensitivity   × 0.3
               + timing        × 0.2
               + 0.2 (constant)
trust_score    = (operational + multi_node) × hardware
               × (0.7 if thermal_throttling else 1.0)

```

Component	What raises it	What lowers it
base_trust	Operator manually marks node as trusted	Default 0.6
uptime_fraction	Node reachable continuously	Network drops, restarts
false_positive_rate	Few unconfirmed-by-other-nodes hits	High solo-hit rate
multi_node_agreement	Hearings corroborated by ≥ 1 other node	Solo observations only
freq_stability_trust	GPSDO-disciplined oscillator (low PPM)	Stock RTL-SDR tuner (50 PPM)
sensitivity_trust	Low noise figure (Airsy R2 at 2.5 dB)	High noise figure (HackRF at 10 dB)
timing_stability_trust	GPSDO + PPS, low offset	NTP only, large offset, stale sync
thermal_throttling	Cool device	Hot phone in sun → 0.7× multiplier

16. The intelligence layer — anomaly flags → threat level → recommended action

Threat level	When it fires	Recommended action
unknown	No flags AND confidence < 0.3	continue_monitoring
low	At least one flag, low severity	continue_monitoring
medium	One high-severity flag OR ≥ 2 medium flags	increase_dwell_time
high	Two high-severity flags OR (one high + confidence ≥ 0.5)	focus_all_nodes (auto-tasks every TDOA-capable node to this frequency)
critical	Any critical-severity flag	alert_operator_immediately — NEVER auto-actioned

Tracks at **high** or **critical** set `escalate_to_atak = true`, which is one of the two gates the CoT emitter checks (§ 17). The frequency-band context labels each track with its regulatory band — Aviation, VHF Public Safety, Marine VHF, UHF Public Safety, ISM 433/915/2.4 GHz, GNSS — so the operator sees 'Emitter at 162.5500 MHz (Marine VHF)' instead of just a raw frequency.

17. ATAK / TAK CoT integration — two-key gate, manual approval queue

17.1 The two-key gate

1. `cot_enabled` (env var `COT_ENABLED=true`, or the toggle in SYS → ATAK / CoT) — the operator-level kill switch.
2. The track's most recent assessment must have `escalate_to_atak = true` (set automatically for high or critical threat levels).

Even an automated critical assessment cannot bypass the operator's cot_enabled flag.

17.2 The manual approval queue (third key for field use)

In the field, set COT_REQUIRE_MANUAL_APPROVAL=true. With this on, escalate_to_atak no longer auto-fires. Each escalation queues at GET /api/v1/approvals and waits for explicit approval before the packet goes out.

Use this in the field. This is the two-person-rule equivalent for a solo operator. A single false-positive can spam the TOC feed otherwise.

17.3 What goes on the wire

CoT 2.0 XML over UDP, defaulting to multicast 239.2.3.1:6969 (the TAK SA feed). For unicast, override COT_DEST_HOST and COT_DEST_PORT.

CoT field	What's in it
type (geolocated emitter)	a-u-G (unknown ground unit) — when there's a TDOA or LOB crosscut fix
type (bearing-only / LOB no crosscut)	b-m-p-s-p-loc (point of interest) — marker placed at the array (node) position
type (sensor node SA beacon)	a-f-G-E-S (friendly ground electronic sensor) — the sensor node's own position beacon, so your nodes appear on the TAK map as sensor icons
point lat / lon	The TDOA or LOB crosscut fix, or the most-trustworthy node's GPS as fallback
point ce (circular error)	Scales 50 m (high confidence) → 5 km (zero confidence). TAK renders this as the circle around the marker.
uid	COT_UID_PREFIX.emitter_id (dot separator)
contact callsign	COT_UID_PREFIX-emitter_id_first_8_chars (dash separator — label visible on ATAK map)
remarks	PREDATOR-RF MHz obs= conf=
element (LOB tracks)	bearing, fov, range, type='LOB', model='KrakenSDR', nodes=N — renders as bearing wedge in ATAK
stale	Default 5 min after emit (COT_STALE_S), then TAK fades the marker

Per-emitter rate limit: 5 seconds between beacons for the same emitter so a chatty source can't flood the TOC.

17.4 Test the path before you go live

```
curl -H "Authorization: Bearer $TOKEN" \
  http://localhost:8000/api/v1/cot/export
```

Returns all tracks that currently pass the escalation gate as CoT XML. Pipe it to a file and import via ATAK's CoT file import — if sensor and emitter markers appear you're wired correctly.

```
# Test a single specific track:
curl -H "Authorization: Bearer $TOKEN" \
  "http://localhost:8000/api/v1/cot/export?emitter_id=<id>"
```

18. AutoTasker — the action loop and its three brakes

When a **high** assessment recommends `focus_all_nodes`, AutoTasker re-tunes every TDOA-capable node to the track's primary frequency. **Critical** assessments are never auto-actioned.

Three brakes prevent runaway tasking:

1. **Per-node rate limit** — 30 s between tunes per node by default (`AUTO_TASKER_MIN_INTERVAL_S`).
2. **Already-tuned check** — skips the tune if the node is already within ± 2 kHz of the requested frequency.
3. **Global per-fleet budget** — at most 30 tunes/min across the entire fleet by default (`AUTO_TASKER_GLOBAL_MAX_PER_MIN`).

AutoTasker is OFF by default (`AUTO_TASKER_ENABLED=false`). Re-tune commands modify the SDR posture, so this is an explicit opt-in.

19. Operator overrides — friendly list, blacklist, manual location

19.1 Friendly list

Mark an `emitter_id` as own-force or known-benign. Effect: suppresses AutoTasker tunes, suppresses CoT escalation, tags the track friendly in the UI. Always recoverable — un-friend in the same UI.

19.2 Frequency blacklist

Add (`start_hz`, `end_hz`) ranges the SweepCoordinator must skip and the TrackManager must drop on ingest. Use cases: noisy commercial broadcaster, known LO leakage, off-limits band.

19.3 Manual location override

Operator supplies a manual lat/lon for an `emitter_id` (confirmed via DF gear or visual). Wins over any TDOA/LOB estimate until cleared. Confidence is pinned to 0.95. All three overrides live in the mission database and rehydrate on restart.

20. Mission lifecycle — start, run, end, export the AAR

20.1 Start

```
TOKEN=$(grep API_BEARER_TOKEN /etc/predator-rf/predator-rf.env | cut -d= -f2)
curl -H "Authorization: Bearer $TOKEN" -X POST localhost:8000/api/v1/missions \
  -d '{"name": "OVERWATCH-20260315", "operator": "K9-Actual"}'
```

Or from the phone: MIS tab → Mission section → *Start mission* → enter name → Start. Starting a new mission auto-ends any in-flight one.

20.2 Check / end / export

```
curl -H "Authorization: Bearer $TOKEN" localhost:8000/api/v1/missions/active
curl -H "Authorization: Bearer $TOKEN" -X POST localhost:8000/api/v1/missions/end
curl -H "Authorization: Bearer $TOKEN" -OJ localhost:8000/api/v1/missions/<id>/export
```

The export bundles a JSONL tarball: every event, track, assessment, approval (approved AND rejected), and override change. Time-stamped and self-contained.

21. Path 2: the Python backend (TOC workstation + RPi sensors)

Two separate programs: predator-rf.service (Python, port 8000) is the intelligence backend — track fusion, TDOA, LOB triangulator, decision engine, CoT. predator-rfd (C++ web backend, port 5555) is the headless sensor daemon for a Linux node acting as a Kujhad Device. They are not interchangeable; they serve different roles.

21.1 Install (one-time)

```
sudo mkdir -p /opt/predator-rf /etc/predator-rf /var/lib/predator-rf/backups
sudo git clone <repo> /opt/predator-rf
cd /opt/predator-rf
sudo python3 -m venv venv
sudo venv/bin/pip install -r requirements.txt
sudo cp deploy/predator-rf.env.example /etc/predator-rf/predator-rf.env
sudoedit /etc/predator-rf/predator-rf.env
sudo cp deploy/predator-rf.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now predator-rf
```

21.2 Key env vars

```
API_BEARER_TOKEN=          # leave blank = open (lab only)
FLEET_NODES=alpha@192.168.1.10:41947:KEY:hackrf,bravo@192.168.1.11:41947:KEY:rtlsdr
TDOA_ENABLED=true
GPS_MAX_AGE_S=60.0
COT_ENABLED=false
COT_DEST_HOST=239.2.3.1
COT_DEST_PORT=6969
COT_UID_PREFIX=PREDATOR
COT_REQUIRE_MANUAL_APPROVAL=false # SET TRUE IN THE FIELD
AUTO_TASKER_ENABLED=false
LOG_LEVEL=INFO
```

Full env-var reference: see docs/linux_web_frontend.md and the annotated predator-rf.env.example file.

21.3 Day-of operations

```
sudo systemctl restart predator-rf
journalctl -u predator-rf -f
curl -s localhost:8000/healthz
curl -s -H "Authorization: Bearer $TOKEN" localhost:8000/api/v1/nodes
curl -s -H "Authorization: Bearer $TOKEN" localhost:8000/api/v1/tracks
```

21.4 KrakenSDR node added to the fleet

A KrakenSDR RPi sensor node integrates into the fleet exactly like any other Kujhad device. The LOB bearings flow through the `kraken_lob_decoder` module → the `DecoderIngestEvent` pipeline → `TrackManager.ingest_lob()` on the backend. No additional configuration is needed beyond the module being enabled and connected to `krakensdr_doa`.

21.5 RNS (Reticulum) transport for CoT

The RNS layer sends the same CoT XML over Reticulum (LoRa, packet radio, I2P, mesh) at the same time as the regular TAK UDP path. Crypto stack: cbor2 envelopes, Argon2id KDF, XChaCha20-Poly1305-IETF AEAD. Opt-in — enabled via the Kujhad panel → RNS Interfaces section. See docs/rns_commanding.md for full configuration.

22. Hardware capability table (every supported SDR)

SDR	Freq range	Max sample	NF	MDS	TDO A	Timing unc.	Price
RTL-SDR Blog v4	25 MHz–1.7 GHz	3.2 MS/s	6.0 dB	–110 dBm	NO	1000 ns	\$40
HackRF One	1 MHz–6 GHz	20 MS/s	10.0 dB	–100 dBm	YES	500 ns	\$340
Airspy R2	24 MHz–1.7 GHz	20 MS/s	2.5 dB	–125 dBm	YES	50 ns	\$170
LimeSDR-USB	100 kHz–3.8 GHz	61.4 MS/s	3.0 dB	–120 dBm	YES (PPS)	100 ns	\$600
bladeRF 2.0	47 MHz–6 GHz	61.4 MS/s	4.0 dB	–118 dBm	YES	80 ns	\$480
ADALM-PLUTO	325 MHz–3.8 GHz	61.4 MS/s	5.0 dB	–115 dBm	YES	200 ns	\$200
SoapySDR generic	1 MHz–6 GHz	10 MS/s	8.0 dB	–105 dBm	NO	500 ns	varies
KrakenSDR	24 MHz–1.7 GHz (×5 channels)	3.2 MS/s per ch	6.0 dB	–110 dBm	LOB DF	n/a (bearing)	\$350

- **For TDOA:** Airspy or LimeSDR — low timing uncertainty AND great sensitivity.
- **HackRF One:** Only SDR in the list that goes below 24 MHz. Acceptable for TDOA. The \$340 upgrade from RTL-SDR.
- **KrakenSDR:** Five-channel coherent DF array. Does LOB, not TDOA. Complements TDOA nodes — one KrakenSDR array can replace two TDOA nodes for a bearing, or combine with TDOA for a tighter hybrid fix.
- **RTL-SDR Blog v4:** No TDOA timing path. Useful as a 4th observer. Also the guts of each KrakenSDR channel.

23. Field-day checklist (print this)

Pre-departure

- Phone(s) charged + power bank packed
- APK version verified — open app → SYS → check version against team's current build
- SDR + antenna + USB-C OTG cable in the bag (one set per phone)
- Baseline file for the area you're going to (record ahead if possible)
- If using ATAK: TAK server reachable + credentials entered + a successful test beacon yesterday
- If using Kujhad fleet: peers added, all on the same overlay, API keys match
- If using KrakenSDR: krakensdr-doa.service starts clean (journalctl -u krakensdr-doa), udev rules written, USB granted
- KrakenSDR heading_deg set correctly — static array = 0; vehicle-mounted = vehicle compass heading
- If running the Linux backend: sudo apt update && sudo apt upgrade -y on workstation + each RPi

- Mission DB backed up: deploy/backup_mission.sh to USB
- System time disciplined: chronyc tracking shows Leap status: Normal
- FLEET_NODES matches today's node serials
- API_BEARER_TOKEN rotated for this mission: openssl rand -hex 32
- COT_REQUIRE_MANUAL_APPROVAL=true if COT_ENABLED=true
- AUTO_TASKER_ENABLED matches your ROE — leave OFF unless authorized to re-tune nodes

Bring-up sequence

1. Power on all sensor nodes; wait 60 s for GPS lock + Kujhad ready
2. If using KrakenSDR RPi: sudo systemctl start krakensdr-doa; confirm green ● Connected in the kraken_lob_decoder panel
3. Operator workstation: sudo systemctl start predator-rf
4. python deploy/preflight.py → must report GO
5. curl http://localhost:8000/healthz → {"status":"ok"}
6. curl -H "Authorization: Bearer \$TOKEN" localhost:8000/api/v1/nodes → every expected node listed
7. Start a mission: POST /api/v1/missions
8. On the operator phone: SPEC → ■ Start. Verify waterfall + GPS green + KUJ green.

In-mission checks (every 30 minutes)

- Glance at thermal indicator on each phone
- Glance at GPS lock on each phone
- curl /metrics → predator_events_total is climbing
- MAP tab → LOB Wedges layer — bearing wedges updating as KrakenSDR array hears signals
- No CoT approvals stuck > 5 min in /api/v1/approvals
- Phone batteries > 20% — swap to power bank before 10%

End of mission

1. Phones: mode back to Manual; export hits to CSV; save fresh baseline if recorded
2. Phones: ■ Stop Listening → close app → unplug SDR
3. KrakenSDR RPi: sudo systemctl stop krakensdr-doa
4. Workstation: POST /api/v1/missions/end
5. GET /api/v1/missions//export → save the AAR tarball
6. deploy/backup_mission.sh → USB

24. Troubleshooting (every symptom we've actually seen)

Symptom	Most likely cause	Fix
Waterfall is black after Start Listening	SDR not selected, or USB permission denied	Source dropdown → re-pick. Unplug + replug SDR → 'Always allow' on dialog.
App says 'device busy'	Another app holds the SDR	Force-stop in Settings → Apps; unplug + replug

GPS never locks	Phone needs sky view; indoor/urban canyon	Step outside; wait 60 s
Letters don't appear when typing in popup	NativeActivity IME composing path	Switch to non-composing keyboard; numeric input always works
Kujhad peer red / disconnected	Network unreachable, port blocked, API keys differ	Ping the peer; verify keys character-for-character
ATAK sensor node markers missing	COT_ENABLED=false, or sensor SA beacon not configured	Enable COT_ENABLED=true. Sensor nodes send a-f-G-E-S SA beacons; check node's CoT config is active.
ATAK emitter marker never appears	Wrong TAK server IP/port, no track has TDOA/LOB fix + escalation	Pull-test: curl -H "Authorization: Bearer \$TOKEN" localhost:8000/api/v1/cot/export — if CoT XML returns, issue is on the TAK side.
ATAK markers stop coming	Approval queue on, operator hasn't approved	Check GET /api/v1/approvals; approve or disable manual approval
KrakenSDR module shows red / Disconnected	krakensdr_doa not running on the RPi, or wrong Host/Port	ssh into the RPi; sudo systemctl start krakensdr-doa; verify port 8082 is reachable
LOB wedge appears but points wrong direction	heading_deg wrong for a vehicle-mounted array	Set heading_deg = vehicle compass heading (true north). Static arrays: heading_deg=0.
Two LOB lines cross but no crosscut dot appears	Crossing angle < 15° (nearly parallel lines — geometrically unstable)	Wait for the array or emitter to move so the angle widens, or wait for a third node
LOB crosscut wildly off	Stale bearing from a previous sweep in the TTL window	Bearings > 30 s old are auto-discarded. If the crosscut is still wrong, suspect heading correction error.
numpy not installed warning in logs	N-LOB WLS path unavailable, falling back to 2-LOB	pip install numpy scipy in the backend venv (or re-run install_rpi.sh --kraken)
TDOA fix never appears on map	< 2 GPS-synced nodes hearing same emission within 5 s	Check /api/v1/nodes — at least 2 must show gps_synchronized=true and gps_age_s < 60
TDOA ellipse is huge (5 km)	Low-confidence fix — only 2 nodes, or all RTL-SDRs	Add more nodes; deploy at least one Airspy or HackRF with GPSDO
Phone hot, framerate drops	Thermal throttling	Get out of sun; reduce sample rate; trust score drops 30% while throttled
401 on every API call	API_BEARER_TOKEN set but no header sent	Add -H "Authorization: Bearer \$TOKEN"

25. Bill of materials — pick your tier

Prices USD, June 2026, ballpark.

Tier 0 — Bare minimum (~\$48)

- Android phone you already own — \$0
- RTL-SDR Blog v4 — \$40
- USB-C OTG adapter — \$8

- Rubber-duck antenna (included with SDR) — \$0

What you can do: Live spectrum, hits, baseline, scans up to 1.7 GHz. No HF, no fleet, no TDOA, no DF.

Tier 1 — Solo field operator (~\$330–540)

- Tier 0 kit — \$48
- Airspy Mini (\$130) or HackRF One (\$340)
- Diamond RH-77CA telescoping whip — \$50
- External USB GPS (BU-353N5) — \$35
- Powered USB-C OTG hub — \$25
- 20,000 mAh USB-C PD power bank — \$40

What you can do: All of Tier 0 plus HF coverage (HackRF), all-day battery, fast GPS, ATAK-ready.

Tier 2 — Solo TOC + one remote sensor (~\$1,400)

- Tier 1 kit (HackRF flavor) — \$540
- Raspberry Pi 5 8 GB + case + cooler + PSU — \$130
- 256 GB A2 microSD — \$25
- Second RTL-SDR Blog v4 (for the Pi) — \$40
- GPS HAT (Adafruit Ultimate GPS HAT) — \$45
- Linux laptop you already own OR add \$500 for one
- Outdoor antenna mount + 25 ft LMR-400 + N-to-SMA pigtails — \$80
- Diamond D130J discone — \$130
- Pelican 1450 case — \$130
- ZeroTier / Tailscale (free tier) — \$0

What you can do: Drop-and-walk sensor node, distributed collection, mission ledger, after-action exports, ONE TDOA pair (phone + RPi).

Tier 3 — Small team with direction finding (~\$6,000–\$11,000+)

- Tier 2 kit — \$1,400
- 3 more RPi sensor nodes (~\$500 each) — \$1,500
- KrakenSDR array + companion RPi 5 — \$350 + \$130 = \$480. Provides single-node LOB DF; a second array gives crosscut fixes.
- At least one Airspy R2 + GPSDO for high-trust TDOA — \$200
- Cellular hotspot + data plan — \$300, OR mesh radio link (goTenna Pro X2 / RAJANT) — \$500–\$5,000
- Commercial directional antennas (Yagi for DF, dipoles per band) — \$400
- 10–25 ft fiberglass tactical mast — \$150
- Pelican / Storm cases per node — \$400
- Operator laptop (rugged ThinkPad / Dell) — \$500–\$2,000

What you can do: Full-perimeter overwatch from one operator screen, real TDOA fixes with tight ellipses, LOB bearing wedges on the map, LOB+TDOA hybrid fixes, ATAK to higher-echelon TOC, ledger across the fleet, real DF capability.

26. Glossary

Term	Meaning
AAR	After-Action Report — the mission ledger export tarball
AutoTasker	The action-loop module that re-tunes nodes based on threat assessments
Baseline	A recorded snapshot of 'normal' RF in an area, used for new-vs-known comparison
CE	Circular Error — the radius around a CoT marker representing 1σ position uncertainty
CoC	Center of Control — backend mode that aggregates from upstream backends (TOC of TOCs)
CoT	Cursor-on-Target — TAK's standard XML-over-UDP situational-awareness wire format
Crosscut	The position fix from two or more LOB bearing lines intersecting
DecisionEngine	The module that turns tracks + anomalies into threat assessments
DOA	Direction of Arrival — the algorithm that converts phase differences between KrakenSDR channels into a bearing
DSD-FME	The P25 P1+P2 decoder bridge
GPSDO	GPS-Disciplined Oscillator — locks the SDR's local oscillator for sub-ppm timing accuracy
LOB	Line of Bearing — a single bearing observation from a direction-finding array. One LOB = a direction. Two LOBs = a crosscut fix.
MDS	Minimum Detectable Signal — the noise-floor sensitivity in dBm. Lower (more negative) is better.
NF	Noise Figure — front-end noise contribution in dB. Lower is better.
OTG	USB On-The-Go — host-mode USB on the phone so it can power and command the SDR
PPS	Pulse-Per-Second — the 1 Hz timing pulse from a GPSDO that disciplines the SDR clock
ROE	Rules of Engagement
SDR	Software-Defined Radio — a radio receiver implemented in software, typically a small USB dongle
TAK	Team Awareness Kit — the SA platform family (ATAK Android, WinTAK Windows, iTAK iOS)
TDOA	Time-Difference-of-Arrival — multi-receiver hyperbolic geolocation technique
TOC	Tactical Operations Center
Track	A fused, persistent identity for an emitter — unique by (frequency, modulation, detecting_node_set)
TTL window	Time-to-live window for LOB measurements (30 s by default); stale bearings are discarded before the solver runs
VFO	Variable Frequency Oscillator — in this app, one of two independent tuners

27. Quick reference card

Action	How to do it
START LISTENING	SPEC tab → source dropdown → ■ Start

DROP A MARKER	Tap a waterfall peak
NAME A MARKER	Tap marker → Assign Marker → tap 'Tap to edit'
RECORD BASELINE	BASE → +Add Range → ■ START RECORDING (5+ min) → Save
START AUTO SCAN	MIS → Mission Mode = Scan → +Add Band → SPEC → ■ Start
ADD KUJHAD PEER	KUJ → Add Peer → Name/Host/Port/Key → Add
PUBLISH ON KUJHAD	KUJ → Listen → port/name/key → toggle Listen
ENABLE ATAK	SYS → ATAK/CoT → server IP+port → Enable CoT (also: COT_REQUIRE_MANUAL_APPROVAL=true in field)
TEST COT EXPORT	curl -H 'Authorization: Bearer \$TOKEN' localhost:8000/api/v1/cot/export
CONNECT KrakenSDR	Module Manager → enable kraken_lob_decoder → set Host/Port → watch for ● Connected
CHECK LOB ON MAP	MAP tab → DF / LOB → toggle LOB Wedges and LOB Crosscut
CHECK HEADING CORRECTION	Static array: heading_deg=0. Vehicle-mounted: heading_deg = vehicle compass heading (true north).
START MISSION	MIS → Start mission → name (or POST /api/v1/missions)
END MISSION	MIS → End mission (or POST /api/v1/missions/end)
EXPORT AAR	GET /api/v1/missions//export

Status / symptom	Action
THERMAL ORANGE	Reduce sample rate, get out of sun
GPS RED	Step outside, wait 60 s
KUJHAD RED	Check network, ping peer, verify API key
KrakenSDR ● Disconnected	ssh into RPi → sudo systemctl start krakensdr-doa → check port 8082
LOB wedge points wrong direction	Check heading_deg — static=0, vehicle=true-north compass heading
LOB crosscut veto (no dot)	Crossing angle < 15° — wait for geometry to improve or add a third node
TDOA NO FIX	Need ≥ 2 GPS-synced nodes hearing same emission < 5 s apart

This document is the contract with the operator. If something here is wrong or out of date, fix it and commit. Rev 2 — June 2026.